Luong Attention
(Global Attention)
(Multiplicative Attention)

Bahdanau Attention
(Additive Attention)

- **Compute Alignment Scores**
- Instead of using **dot product $\boldsymbol{s^T h_i}$** (as in Luong), we now use **a neural network to compute alignment scores**:

$$e_i = v^T \tan h(W_s s + W_h h_i)$$

Assuming some learned weight matrices W_s, W_h, we get:

$$e_1 = v^T \tanh([0.9, 0.5, -0.4])$$

Let's assume v^T is a learned vector that gives:

$$e_1 = 0.5$$

Score for "love" (h_2)

$$e_2 = v^T \tanh(W_s[0.8, 0.2, -0.5] + W_h[0.7, 0.6, -0.1])$$

Assuming some learned weight matrices:

$$e_2 = v^T \tanh([1.2, 0.8, -0.3]) = 1.5$$

Score for "NLP" (h_3)

$$e_3 = v^T \tanh(W_s[0.8, 0.2, -0.5] + W_h[0.2, 0.4, 0.3])$$

Assuming some learned weights:

$$e_3 = v^T \tanh([1.0, 0.6, 0.1]) = 1.0$$

3️⃣ Apply Softmax to Get Attention Weights

Now, we normalize the scores using the **softmax function**:

$$\alpha = \text{softmax}([0.5, 1.5, 1.0])$$

$$\alpha = [0.2, 0.5, 0.3]$$

4️⃣ Compute Attention-Weighted Context Vector

The **context vector** is a weighted sum of the encoder hidden states:

$$c = (0.2 \times h_1) + (0.5 \times h_2) + (0.3 \times h_3)$$

Substituting values:

$$c = (0.2 \times [0.1, 0.3, -0.2]) + (0.5 \times [0.7, 0.6, -0.1]) + (0.3 \times [0.2, 0.4, 0.3])$$

$$c = [0.02, 0.06, -0.04] + [0.35, 0.3, -0.05] + [0.06, 0.12, 0.09]$$

$$c = [0.43, 0.48, 0.00]$$

The **context vector** c is now used to help predict "J'aime".

Step 4: Generating the Next Words ("le NLP")

- The decoder updates its **hidden state** s' using the new context vector.
- It computes a **new set of attention scores**.
- This time, attention may shift more toward "NLP", helping correctly generate **"le NLP"**.

Step 5: Final Translation Output

After iterating through each decoding step, we get the final translation:

$$\text{"I love NLP"} \quad \Rightarrow \quad \text{"J'aime le NLP"}$$

✔ **The attention dynamically shifts focus to different words at each step.**
✔ **Bahdanau Attention allows more flexibility in learning attention weights.**

- **Step 1: Encode the input sentence**
- Input: "I love you"
- The encoder processes each word and outputs a sequence of hidden states:

```
encoder_outputs = [h1, h2, h3] # one hidden state per word
```

- **Step 2: Initialize the decoder**
- Use the **last encoder hidden state** (e.g. h3) to initialize the decoder's hidden state:

$$s_0 = h3 \text{ \# or a transformation of it}$$

- **Step 3: At each decoding time step (t = 1, 2, 3...)**
- Repeat the following steps to generate one target word at a time:

- **Step 3.1: Compute alignment scores**
- For each encoder hidden state h_i, compute a score with the decoder's current hidden state s_t:

$$score_i = V^t * \tanh(W_1 * h_i + W_2 * s_t)$$

This gives a **score for each input word** — higher score = more relevant.

- **Step 3.2: Compute attention weights**
- Apply **softmax** to all the scores:

$$\text{attention_weights} = \text{softmax}([\text{score}_1, \text{score}_2, ..., \text{score_n}])$$

This gives a **distribution** over the input words (e.g., [0.1, 0.2, 0.7]).

- **Step 3.3: Compute the context vector**
- Use the attention weights to get a **weighted sum** of the encoder outputs:

$$\text{context_vector} = \Sigma\,(\text{attention_weight_i} * \text{h_i})$$

This context tells the decoder **what to focus on** from the input.

- **Step 3.4: Generate the next word**
- Use the **context vector** and **decoder hidden state** to predict the next word:

  output = softmax(Dense(concat(context_vector, s_t)))

- **Step 3.5: Update decoder hidden state**
- Use the generated word (or ground truth during training) as input to the decoder to compute the **next hidden state** s_{t+1} and repeat.

Bahdanau vs. Luong Attention: Key Differences

1. Scoring Function

- **Bahdanau Attention**: Uses an additive scoring function, which involves a feedforward neural network. This method is more flexible and can learn more complex relationships between the encoder and decoder states but at the cost of higher computational complexity.
- **Luong Attention**: Uses a multiplicative or dot-product scoring function, which is computationally efficient and simpler to implement. However, it may not capture as complex relationships as the additive method.

2. Complexity

- **Bahdanau Attention**: More complex due to the additional parameters in the feedforward network used for scoring. This can make training slower but can potentially lead to better performance on complex tasks.
- **Luong Attention**: Simpler and faster to compute, making it more suitable for large-scale or real-time applications where computational efficiency is critical.

3. Flexibility

- **Bahdanau Attention**: Offers greater flexibility due to its learnable parameters in the scoring function. This allows the model to adapt better to various tasks, particularly those requiring complex alignment.
- **Luong Attention**: While less flexible, Luong attention is easier to tune and integrate into existing models, making it a popular choice for many applications.

4. Implementation

- **Bahdanau Attention**: Typically requires more code and understanding to implement due to its complexity. It also requires careful tuning of the neural network used in the scoring function.
- **Luong Attention**: Easier to implement, especially in frameworks that already support dot-product operations. It is often preferred in practical applications where ease of use and efficiency are priorities.

Feature	Luong Attention (Multiplicative)	Bahdanau Attention (Additive)
Score Computation	Dot product: $s^T h_i$	Neural network: $v^T \tanh(W_s s + W_h h_i)$
Computational Cost	Faster (fewer operations)	Slower (extra neural net layer)
Context Vector	Weighted sum of encoder states	Weighted sum of encoder states
Best For	Shorter sequences	Longer, complex sequences

Attention Mechanisms

- The concept of attention in DL emerged from the applications of NLP coupled with machine translation.
- Alex Graves, a lead AI research scientist at DeepMind, the renowned AI research collective, indirectly defined attention

Attention is memory per unit of time.

- This mechanism is particularly crucial in tasks where the context is vital for prediction, such as natural language processing (NLP), image recognition, and more. The attention mechanism was developed to address the challenge of handling long sequences of data where not all parts are equally relevant

- Traditional neural networks, especially those dealing with sequences like Recurrent Neural Networks (RNNs) and long short-term memory (LSTM) networks, tend to lose information over long distances, making it hard for them to remember and use earlier parts of the data.
- This is problematic in tasks like language translation, where understanding the beginning of a sentence is crucial for accurately translating the end.

- Attention allows a model to “remember” and focus on different parts of the input sequence, regardless of their position, by assigning different weights to different parts of the data. This means that instead of treating all the data equally, the model can learn to pay more attention to the bits that are more relevant to the task it’s performing

Types of Attention Mechanism

1. **Soft Attention**
2. **Hard Attention**
3. **Self-Attention**
4. **Multi-Head Attention**

Soft Attention

- Soft attention provides a differentiable mechanism that allows models to focus on different parts of the input with varying degrees of emphasis, but smoothly and without making hard decisions.
- In other words, every part of the input gets some degree of attention, but some parts get more than others.

Soft Attention

- An example use case is in image captioning, where the model needs to focus on different parts of an image when generating each word in the caption. By softly attending to regions of the image, the model can generate more accurate and contextually relevant captions.

Hard Attention

- Hard attention, in contrast to soft attention, makes a discrete choice about which part of the input to attend to. It’s like turning a spotlight on one area while ignoring the others completely.
- This mechanism is not differentiable, which makes it harder to train using standard backpropagation techniques, often requiring reinforcement learning or other methods.

Hard Attention

- A use case for hard attention is in visual question answering, where the model needs to focus on a specific part of an image to answer a question correctly. By selecting only the relevant region, the model can improve its accuracy in providing the right answer.

Self-Attention

- Self-attention layers allow elements in an entire sequence to attend to all other elements in the same sequence. This is particularly powerful in tasks where the context is crucial, and the relationship between different elements is key to understanding the whole.

Self-Attention

- For example, in a sentence, the meaning of a word can depend on the other words around it, not just the ones directly adjacent. Self-attention is a core component of the Transformer architecture, which has been revolutionary in fields like natural language processing.
- Applications include machine translation, where understanding the context and the interplay between words in a sentence can significantly enhance translation quality.

Multi-Head Attention

- Multi-head attention mechanism is an extension of self-attention where the mechanism is applied several times in parallel. The key idea is that each “head” can focus on different parts of the input, capturing various aspects of the information. The attention outputs of these heads are then combined, giving the model a richer representation of the input.

Advantages of Attention Mechanism

- **Improved Contextual Understanding:** Attention mechanisms enhance models ability to understand context. This is particularly beneficial in tasks like language translation, where the meaning of words can depend heavily on their context.
- **Enhanced Long-Sequence Handling:** Traditional RNNs and even LSTMs can struggle with long sequences due to issues like vanishing gradients. Attention allows models to directly focus on relevant parts of the input sequence, mitigating some of these issues and improving performance on tasks involving long inputs.

Advantages of Attention Mechanism

- **Increased Model Interpretability:** By visualizing the attention weights, researchers can gain insights into how the model is making its decisions, which parts of the input are influencing the output, and how different components of the data interact. This can help with debugging and improving model designs.
- **Flexibility and Applicability:** Attention mechanisms are versatile and can be incorporated into various types of neural networks, offering improved performance across a broad spectrum of tasks.

Disadvantages of Attention Mechanism

- **Computational Complexity:** Calculating attention weights can be computationally intensive, especially for large models and datasets. This can increase training times and the computational resources required, which may not be feasible in all scenarios.
- **Overfitting Risk:** Increased complexity can lead to overfitting, causing the model to learn noise or peculiarities in the training data, resulting in poor generalization on unseen data.

Disadvantages of Attention Mechanism

- **Implementation Complexity:** Integrating attention into a model can add to the complexity of the model's architecture and its implementation, requiring careful tuning and potentially increasing the time needed for model development and debugging.
- **Scalability Issues:** As the entire sequence length increases, the computational cost and memory requirements of attention mechanisms can grow quadratically, which can be a significant bottleneck for tasks involving very long sequences.